



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Lucas Pereira da Silva

Refatoração Automática de Código de Teste

Florianópolis
20 de junho de 2026

SUMÁRIO

1	INTRODUÇÃO	2
	REFERÊNCIAS	6

1 INTRODUÇÃO

Teste de software é uma das atividades presentes no processo de desenvolvimento e manutenção de software. Conforme o *Guide to the Software Engineering Body of Knowledge* (SWEBOK) (Bourque; Fairley et al., 2014) define, teste de software consiste na verificação dinâmica de que um programa provê um comportamento esperado a partir de um conjunto finito de casos de teste, adequadamente selecionados a partir de um domínio de execução potencialmente infinito. Por muito tempo, teste de software foi visto sobretudo como uma etapa de verificação da qualidade do software produzido, e não como meio de construir um produto com qualidade (Murugesan, 1994). Nesse sentido, testes eram frequentemente realizados apenas ao final da etapa de codificação (Yang et al., 2011), com o objetivo de detectar falhas e fornecer um indicativo de que o software funciona conforme o esperado (Bertolino, 2007). Com o passar do tempo, a percepção sobre a utilidade do teste evoluiu. A atividade deixou de ser vista como etapa posterior ao desenvolvimento e passou a estar presente durante todo o processo de desenvolvimento e manutenção de software (Bourque; Fairley et al., 2014). Meszaros (2007) considera a atividade de teste parte intrínseca do processo de desenvolvimento de software. Assim, a atividade de teste de software passou a incorporar propósitos variados, como detectar falhas (Tiwari; Goel, 2013), prevenir que erros sejam introduzidos (Beizer, 1990), prover um indicativo de que o software funciona (Bertolino, 2007), auxiliar na compreensão do projeto e do código do sistema (Freeman; Pryce, 2009) e servir como meio para especificação (Alvestad, 2007) e documentação (Haugset; Hanssen, 2008) dos requisitos.

Nesse contexto, o código de teste deixou de ser um artefato auxiliar e passou a exigir os mesmos cuidados aplicados ao código de produção. Assim como o código da aplicação, o código de teste precisa ser mantido, compreendido e ajustado durante todo o projeto (Greiler; Deursen; Storey, 2013b). Para Meszaros (2007), a atividade de teste não deveria aumentar o esforço total de desenvolvimento e manutenção do software, uma vez que o esforço gasto com essa atividade deve ser compensado pela melhoria da qualidade do software. Entretanto, manter o código de teste apresenta desafios, principalmente quando a codificação e a evolução dos testes são realizadas diretamente pelo desenvolvedor. Conforme o software evolui e a quantidade de código aumenta, torna-se mais difícil manter os testes com consistência e sem perder o controle sobre eles. Fenômeno semelhante ocorre no código de produção: sem refatoração contínua, cada nova funcionalidade tende a exigir mais esforço do que a anterior (Fowler, 1999). Com os testes, observa-se comportamento análogo. Novos testes passam a ser mais difíceis de adicionar e a duplicação tende a aumentar (Berner; Weber; Keller, 2005). Isso pode levar à degradação da manutenibilidade do código de teste (Greiler; Deursen; Storey, 2013a). Segundo Emery (2009), um motivo comum para o abandono de testes automatizados é o fato de os testes se tornarem frágeis e de alto custo de manutenção, o que pode tornar os desenvolvedores reticentes tanto em adicionar novos testes quanto em executar os existentes. Mesmo em um cenário onde tanto o código de produção quanto o código de teste são gerados por modelos de Inteligência Artificial (IA), a manutenibilidade do código de teste é um problema que precisa ser tratado. Shamshiri

et al. (2018) mostra que testes gerados automaticamente afetam o tempo e a confiança do desenvolvedor, tendo as tarefas de manutenção levado 29% mais tempo quando realizadas com testes gerados automaticamente. Nesse sentido, independente do contexto, a manutenibilidade do código de teste ainda é um aspecto importante no processo de desenvolvimento de software.

Evitar a duplicação no código de teste é uma das formas de melhorar a manutenibilidade. Isso porque mudanças na aplicação passam a ter impacto menor nos testes, e o desenvolvedor dispõe de menos código para manter e gerenciar (Berner; Weber; Keller, 2005; Greiler; Deursen; Storey, 2013a). De acordo com Berner, Weber e Keller (2005), testes tendem a ser repetitivos e apresentam alto potencial de reuso; conforme o software evolui e o número de testes aumenta, é comum que a quantidade de código duplicado entre os testes também aumente. As Figuras 1 e 2 ilustram uma duplicação do código de teste. Cada figura apresenta um método de teste em uma classe distinta. Os exemplos utilizam a linguagem Kotlin e o framework JUnit 5. O código das linhas 3 a 8 em ambos os testes configura o sistema em teste (*System Under Test* – SUT) da mesma forma, o que produz código de configuração duplicado.

```

1 @Test
2 fun 'deve cancelar a matrícula do estudante'() {
3     val ufsc = Universidade("UFSC")
4     val alice = ufsc.admitir("Alice")
5     val bob = ufsc.contratar("Bob")
6     val cc = ufsc.criarPrograma("Ciência da Computação")
7     val algoritmos = cc.criarDisciplina("Algoritmos", bob)
8     algoritmos.matricular(alice)
9
10    val sucesso = algoritmos.cancelarMatricula(alice)
11
12    assertTrue(sucesso)
13    assertTrue(algoritmos.estudantes.isEmpty())
14 }

```

Figura 1 – Método de teste com configuração inline do SUT, suscetível a duplicação de código

Fonte: Elaborado pelo autor.

```

1 @Test
2 fun 'não deve matricular o estudante duas vezes'() {
3     val ufsc = Universidade("UFSC")
4     val alice = ufsc.admitir("Alice")
5     val bob = ufsc.contratar("Bob")
6     val cc = ufsc.criarPrograma("Ciência da Computação")
7     val algoritmos = cc.criarDisciplina("Algoritmos", bob)
8     algoritmos.matricular(alice)
9
10    val sucesso = algoritmos.matricular(alice)
11
12    assertFalse(sucesso)
13    assertEquals(listOf(alice), algoritmos.estudantes)
14 }

```

Figura 2 – Método de teste com lógica de configuração do SUT idêntica à da Figura 1, ilustrando a duplicação

Fonte: Elaborado pelo autor.

Técnicas que reduzam essa duplicação podem contribuir para preservar a manutenibilidade do código de teste ao longo do projeto, bem como diminuir o esforço total de desenvolvimento dos testes. Uma dessas técnicas é a configuração implícita (ou *implicit setup*), em que os testes são organizados em classes de teste, de forma que o código de configuração comum possa ser centralizado em um método de configuração (Meszaros, 2007). A Figura 3 ilustra a aplicação da configuração implícita. Para além da remoção da duplicação de código, esse exemplo mostra como a utilização da configuração implícita traz mais clareza para a compreensão do teste. O contexto comum (ter um aluno matriculado em uma disciplina) aparece apenas uma vez e cada método de teste foca apenas no aspecto mais importante para aquele teste.

```

1 class TesteUniversidade {
2     private lateinit var algoritmos: Disciplina
3     private lateinit var alice: Estudante
4
5     @BeforeEach
6     fun configurar() {
7         val ufsc = Universidade("UFSC")
8         alice = ufsc.admitir("Alice")
9         val bob = ufsc.contratar("Bob")
10        val cc = ufsc.criarPrograma("Ciência da Computação")
11        algoritmos = cc.criarDisciplina("Algoritmos", bob)
12        algoritmos.maticular(alice)
13    }
14
15    @Test
16    fun 'deve cancelar a matrícula do estudante'() {
17        val sucesso = algoritmos.cancelarMatricula(alice)
18
19        assertTrue(sucesso)
20        assertTrue(algoritmos.estudantes.isEmpty())
21    }
22
23    @Test
24    fun 'não deve matricular o estudante duas vezes'() {
25        val sucesso = algoritmos.maticular(alice)
26
27        assertFalse(sucesso)
28        assertEquals(listOf(alice), algoritmos.estudantes)
29    }
30 }

```

Figura 3 – Suíte de testes refatorada com configuração implícita, centralizando a lógica comum de inicialização e eliminando a duplicação

Fonte: Elaborado pelo autor.

Entretanto, nem sempre é simples decidir como os testes devem ser organizados. Em projetos com muitos testes, a decisão de como distribuir os testes entre as classes de teste pode ser complexa. Isso porque cada teste pode ter requisitos de configuração diferentes, o que pode levar a uma explosão combinatorial de possibilidades. Por exemplo, considere o teste da Figura 4. Esse teste requer um estado do SUT diferente do estado comum dos testes da Figura 3. Ao invés de um aluno matriculado, esse teste requer uma disciplina fechada. Como nem toda configuração pode ser compartilhada, o teste deve ser colocado em uma classe de teste diferente ou o código que configura a matrícula do aluno na disciplina deve ser removido da configuração

implícita e duplicado entre os dois testes que dependem dela.

```
1 @Test
2 fun 'não deve matricular estudante em disciplina fechada'() {
3     val ufsc = Universidade("UFSC")
4     val alice = ufsc.admitir("Alice")
5     val bob = ufsc.contratar("Bob")
6     val cc = ufsc.criarPrograma("Ciência da Computação")
7     val algoritmos = cc.criarDisciplina("Algoritmos", bob)
8     algoritmos.fechar()
9
10    val sucesso = algoritmos.matricular(alice)
11
12    assertFalse(sucesso)
13    assertTrue(algoritmos.estudantes.isEmpty())
14 }
```

Figura 4 – Método de teste que requer um estado distinto do SUT, impedindo o reuso da configuração comum da Figura 3

Fonte: Elaborado pelo autor.

O problema fica claro mesmo com um exemplo trivial: como organizar os métodos de teste levando em conta a redução da duplicação de código? Nesse exemplo, vale a pena manter o novo teste na mesma classe dos demais? Por um lado, o código do novo teste que é comum aos demais deixaria de ser duplicado. Por outro lado, o código que matricula o aluno na disciplina precisaria ser duplicado entre os dois testes que dependem dele. Nesse exemplo, se formos considerar apenas o reuso de código, ainda é claro que valeria a pena mover o novo teste para a mesma classe de teste dos demais, uma vez que o reuso obtido seria maior que a duplicação introduzida. Entretanto, em um ambiente real de desenvolvimento, com centenas ou milhares de testes, a decisão de como distribuir os testes entre as classes de teste passa a ser complexa e com uma explosão combinatorial de possibilidades.

REFERÊNCIAS

- ALVESTAD, K. **Domain Specific Languages for Executable Specifications**. Dissertação (Mestrado) — Institutt for datateknikk og informasjonsvitenskap, 2007.
- BEIZER, B. **Software Testing Techniques**. 2. ed. [S.l.]: Van Nostrand Reinhold Company, 1990.
- BERNER, S.; WEBER, R.; KELLER, R. K. Observations and lessons learned from automated testing. In: **Proceedings of the 27th International Conference on Software Engineering**. [S.l.]: ACM, 2005. p. 571–579.
- BERTOLINO, A. Software testing research: Achievements, challenges, dreams. In: **2007 Future of Software Engineering**. [S.l.]: IEEE Computer Society, 2007. p. 85–103.
- BOURQUE, P.; FAIRLEY, R. E. et al. **Guide to the software engineering body of knowledge (SWEBOK): Version 3.0**. [S.l.]: IEEE Computer Society Press, 2014.
- EMERY, D. H. Writing maintainable automated acceptance tests. In: **Agile Testing Workshop, Agile Development Practices**. Orlando, Florida: [s.n.], 2009.
- FOWLER, M. **Refactoring: Improving the Design of Existing Code**. [S.l.]: Addison-Wesley, 1999.
- FREEMAN, S.; PRYCE, N. **Growing Object-Oriented Software, Guided by Tests**. [S.l.]: Pearson Education, 2009.
- GREILER, M.; DEURSEN, A. van; STOREY, M.-A. Automated detection of test fixture strategies and smells. In: **2013 IEEE Sixth International Conference on Software Testing, Verification and Validation**. [S.l.]: IEEE, 2013. p. 322–331.
- GREILER, M.; DEURSEN, A. van; STOREY, M.-A. Strategies for avoiding text fixture smells during software evolution. In: **Proceedings of the 10th Working Conference on Mining Software Repositories**. [S.l.]: IEEE Press, 2013. p. 387–396.
- HAUGSET, B.; HANSSEN, G. K. Automated acceptance testing: A literature review and an industrial case study. In: **Agile 2008 Conference**. [S.l.]: IEEE, 2008. p. 27–38.
- MESZAROS, G. **xUnit test patterns: Refactoring test code**. [S.l.]: Pearson Education, 2007.
- MURUGESAN, S. Attitude towards testing: a key contributor to software quality. In: **Proceedings of 1994 1st International Conference on Software Testing, Reliability and Quality Assurance (STRQA'94)**. [S.l.: s.n.], 1994. p. 111–115.
- SHAMSHIRI, S. et al. How do automatically generated unit tests influence software maintenance? In: **2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)**. [S.l.: s.n.], 2018. p. 250–261.
- TIWARI, R.; GOEL, N. Reuse: reducing test effort. **ACM SIGSOFT Software Engineering Notes**, v. 38, n. 2, p. 1–11, 2013.
- YANG, Y. et al. Testqual: Conceptualizing software testing as a service. **e-Service Journal**, Indiana University Press, v. 7, n. 2, p. 46–65, 2011. ISSN 15288226, 15288234. Disponível em: <http://www.jstor.org/stable/10.2979/eservicej.7.2.46>.